System Programming - Homework 1

GEBZE TECHNICAL UNIVERSITY

Prepared by:

Dicle Çoban

13.03.2026

Contents

Abstract **ii**

Introduction **1**

0.1 Design 1

0.1.1 Data Structure 1

0.1.2 Program Flow 2

0.2 Implementation 2

0.2.1 Directory Traversal 2

0.2.2 Filename Matching with + Operator 3

0.2.3 Permission Matching 3

0.2.4 Tree Output 3

0.2.5 Signal Handling 3

0.3 Requirements 3

0.3.1 Satisfied Requirements 3

0.3.2 Not Satisfied Requirements 4

0.4 Conclusion 4

0.5 Test Results 4

Abstract

This report presents the design and implementation of an advanced file search program for POSIX compatible operating systems. The program recursively traverses a given directory and searches for files matching specified criteria including filename, file size, file type, permissions, and number of links.

Introduction

This homework implements an advanced file search program for POSIX compatible operating systems, similar to the Unix `find` command. The program recursively traverses a given directory and searches for files that match the specified criteria. The search criteria include filename, file size, file type, permissions, and number of links. The results are printed in a nicely formatted tree structure.

0.1 Design

0.1.1 Data Structure

A `struct criteria` is used to store the search parameters provided by the user. It contains the following fields:

- `path` — the directory to search in
- `filename` — the filename pattern to match
- `size` — the file size in bytes
- `type` — the file type character
- `permissions` — the permission string (e.g. `rwxr-xr--`)
- `nlinks` — the number of hard links

Fields that are not provided by the user are initialized to `NULL` (If it is pointer char), `\0` (If it is char), or `-1` (If it is integer) to indicate that they should not be checked.

Function	Description
`init_criteria()`	Initializes all fields of the criteria struct to default values
`parse_criteria()`	Parses command-line arguments using `getopt()` and fills the criteria struct
`search()`	Recursively traverses the directory tree
`check_criteria()`	Checks whether a file matches all given criteria
`match_filename()`	Matches a filename against a pattern supporting the `+` operator
`match_size()`	Checks if the file size matches
`match_type()`	Checks if the file type matches
`match_permissions()`	Checks if the file permissions match
`match_nlinks()`	Checks if the number of links matches
`print_result()`	Prints the file path in tree format
`signal_handler()`	Handles CTRL-C signal

Table 1: Functions and their descriptions

0.1.2 Program Flow

The program starts in `main()`, which registers the signal handler, initializes and fills the criteria struct, and then calls `search()`. The `search()` function opens the directory, reads each entry, calls `lstat()` to get file metadata, and passes the results to `check_criteria()`. If all criteria match, `print_result()` is called. If the entry is a directory, `search()` is called recursively.

0.2 Implementation

0.2.1 Directory Traversal

The program uses `opendir()` and `readdir()` to traverse directories recursively. `lstat()` is used to avoid infinite loops. The entries `.` and `..` are skipped using `strcmp()`. For each file, the full path is constructed using `snprintf()`.

0.2.2 Filename Matching with + Operator

The `+` operator means one or more occurrences of the previous character. The implementation uses two pointers, `pattern` and `string`, to traverse both strings at the same time. When a `+` is detected after the current pattern character, the string pointer advances as long as the characters match. When they no longer match, the pattern advances past the `+`. The matching is case insensitive, implemented using `tolower()`.

0.2.3 Permission Matching

The user provides permissions as a 9-character string such as `rwxr-xr--`. The `parse_permission()` function converts this string into a binary array where `1` means the bit should be set and `0` means it should not. The `match_permission()` function then compares each bit against the file's `st_mode` using bitwise AND with POSIX permission macros such as `S_IRUSR`, `S_IWGRP`, etc.

0.2.4 Tree Output

The depth of each file in the directory tree is tracked using the `depth` parameter passed through recursive calls. The number of dashes printed before the filename is calculated as `depth * 4 - 2`.

0.2.5 Signal Handling

The program registers a handler for `SIGINT` using `signal()`. When CTRL-C is pressed, the handler prints an informative message to `stderr` and exits cleanly.

0.3 Requirements

0.3.1 Satisfied Requirements

- Recursive directory traversal
- Filename matching with `+` operator (case insensitive)
- File size matching

- File type matching
- Permission matching
- Number of links matching
- Tree formatted output
- CTRL-C signal handling
- Error messages printed to `stderr`
- Usage information printed when `-w` is missing
- "No file found" message when no results are found
- Makefile provided

0.3.2 Not Satisfied Requirements

All requirements have been successfully implemented.

0.4 Conclusion

This homework provided hands-on experience with POSIX system programming in C. The program successfully implements an advanced file search tool that traverses directory trees recursively and filters files based on multiple criteria. The most challenging part was implementing the `+` regular expression operator without using any library functions but it handled successfully with pointers.

0.5 Test Results

The following tests were conducted to verify the correctness of the program.

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir
|--subDir1
|------lostttfile
|------subDir2
|----------world.txt
|--hello.txt
|--testfile100
```

Figure 1: Basic directory traversal output

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -f "lost+file"
|------lostttfile
```

Figure 2: Filename matching with + operator

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -t f
|------lostttfile
|----------world.txt
|--hello.txt
|--testfile100
```

Figure 3: File type filtering - regular files only

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -t d
|--subDir1
|------subDir2
```

Figure 4: File type filtering - directories only

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -b 100
|--testfile100
```

Figure 5: File size filtering - 100 bytes

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -p "rwxr-xr-x"
|--subDir1
|------subDir2
|--hello.txt
```

Figure 6: Permission filtering

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -l 1
|------lostttfile
|----------world.txt
|--hello.txt
|--testfile100
```

Figure 7: Number of links filtering

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -f "lost+file" -b 100 -t f
No file found!
```

Figure 8: Multiple criteria combination - no matching file

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -t f -l 1
|------lostttfile
|----------world.txt
|--hello.txt
|--testfile100
```

Figure 9: Multiple criteria - file type and number of links

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -w testDir -f "xyz"
No file found!
```

Figure 10: No file found message

```
diclesaracoban@Dicle-MacBook-Pro HW1 % ./myFind -f "hello"
Usage: ./myFind -w path [-f filename] [-b size] [-t type] [-p permissions] [-l nlinks]
```

Figure 11: Usage information when -w flag is missing